Disaster Tweets: Real or Not?

</> Code

A worked case study in text classification, and a case for choosing a model on more than one metric

AUTHOR
Your Name

PUBLISHED
September 29, 2026

Interactive application

Open the Streamlit app

1 Introduction

I built this project to predict whether a tweet describes a **real disaster** (`"Disaster"`) or **not** (`"Not Disaster"`) from its raw text, `keyword`, and `location` fields — the classic "Real or Not? NLP with Disaster Tweets" task. I picked it because text classification and social-media monitoring felt like a genuinely different problem from the tabular datasets I'd worked with before, and I wanted to see how much of what I'd learned about imbalanced classes and model comparison would still hold up when the features are words instead of numbers.

Getting the data wasn't as simple as I expected. The original Kaggle *competition* (`nlp-getting-started`) requires accepting its rules on the website before the API will serve anything — I checked directly, and `kaggle competitions download` returns a 403 even when I was authenticated. I ended up using a public *dataset* mirror of the same task instead, `vstepanenko/disaster-tweets`, which has the same 4 input columns and a larger, re-scraped sample.

Beyond just running the usual "compare class-imbalance techniques" playbook, I wanted this project to make one point I hadn't seen spelled out clearly elsewhere: the model that scores highest on cross-validation isn't automatically the right one to put into production. Below, the single best-measured model I found turned out to be practically impossible to explain with SHAP, and choosing a slightly-lower-scoring model instead was a deliberate tradeoff I made on purpose, not an accident or an oversight.

2 Data

▶ Code

```
11370 rows, 5 columns

target
0    0.814072
1    0.185928
Name: proportion, dtype: float64
```

2.1 Real data-quality issues I found by checking, not assuming

I didn't take the raw data at face value, and it's a good thing I didn't — checking it directly turned up three real problems:

- **Exact-duplicate tweets, some with conflicting labels**: I found 147 duplicate tweet texts, 2 of which carry *different* labels for the same text — genuine annotation disagreement, presumably from two different human labelers disagreeing. Left alone, duplicates could leak across a train/validation split: the model would "predict" a duplicate tweet in the validation set by memorizing the identical copy it saw during training, which inflates the validation score without teaching the model anything that generalizes to a genuinely new tweet. The 2 conflicting-label cases can't be trusted either way, so I dropped them rather than guess which label was right.
- `keyword` **contains raw URL-encoding artifacts**: values like `"mass%20murder"` instead of `"mass murder"` — the underlying scrape never decoded the URL-encoded parameter. This is the kind of bug that silently degrades a categorical feature if you don't look at the actual values, since `mass%20murder` and `mass murder` would be treated as two unrelated categories instead of one.
- `location` **is too sparse and inconsistent to use as a categorical feature**: 4,504 distinct free-text values across 7,952 non-null rows, with `"USA"`, `"United States"`, and `"US"` all counted separately. One-hot encoding this as-is would create thousands of near-useless, mostly-empty columns, so I reduced it to a simpler signal instead (see Feature engineering below).

▶ Code

```
11221 rows after cleanup (from 11370 raw)

target
Not Disaster    0.815614
Disaster        0.184386
Name: proportion, dtype: float64
```

3 Feature engineering

Text needed a genuinely different feature-engineering approach than the tabular datasets I'd worked with before — I couldn't just reuse a scaler and a one-hot encoder, since there's no "numeric column" to scale in a tweet:

▶ Code

	cleaned_text	has_location	text_length	word_count
0	communal violence in bhainsa telangana stones ...	0	125	19
1	telangana section has been imposed in bhainsa ...	0	131	23
2	arsonist sets cars ablaze at dealership	1	63	7

- **TF-IDF** (word uni+bigrams, up to 5,000 features) on cleaned text. TF-IDF (term frequency–inverse document frequency) turns each tweet into a vector of word/phrase weights: a word gets a high weight in a tweet if it appears often *in that tweet* but rarely across the rest of the dataset, which is exactly what makes a word a useful signal for telling tweets apart. I stripped URLs and `@mentions` first, since their specific characters carry no signal, and unwrapped hashtags to their word content so `#earthquake` contributes the word "earthquake" rather than a token that's technically different from every plain-text mention of the same word.
- **Hand-crafted numeric signals**: length, word count, and URL/mention/hashtag counts. These are cheap to compute — just counting characters and substrings — and I almost skipped them as too simple to matter. That assumption turned out to be wrong: as the interpretability section below shows, they end up mattering more than almost any individual word in the vocabulary.
- `has_location` instead of raw location text, given the cardinality problem above — a single yes/no flag for "was any location given at all" is a weaker signal than a clean location would be, but it's a real signal I could actually trust, unlike the raw free-text field.

4 Methodology

4.1 Why macro-F1, made concrete

▶ Code

```
Accuracy: 0.8775
Macro-F1: 0.7458
```

Accuracy (~87.8%) looked strong to me at first glance, but that's an illusion caused by the ~81.6% majority ("Not Disaster") share — a model that predicted "Not Disaster" for every single tweet,

with zero understanding of the text, would already score close to that number. Macro-F1 (which averages the F1-score of each class equally, instead of weighting by how common the class is) is noticeably lower here, and that gap is the real story: it exposes that the Disaster class isn't being caught nearly as well as the headline accuracy number suggests. That's why I use macro-F1, not accuracy, for every comparison that follows.

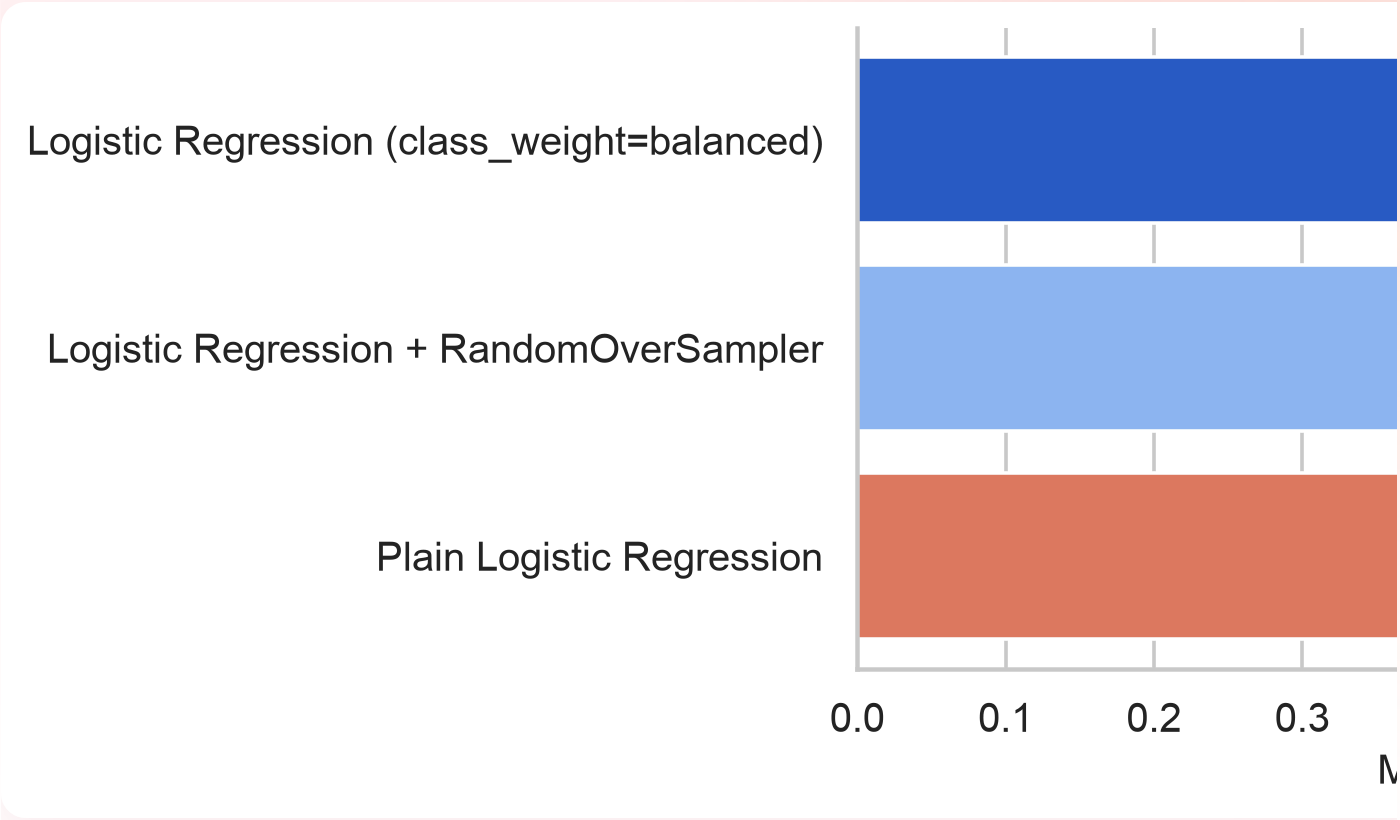
4.2 Class-imbalance handling, compared honestly

Class imbalance means one class ("Not Disaster") vastly outnumbers the other, which tends to bias a classifier toward predicting the majority class by default, since that's the easiest way to minimize error on average. Two common fixes are **class-weighting** (telling the model to penalize a mistake on the minority class more heavily during training) and **oversampling** (giving the model more copies of the minority class to learn from). I'd have used **ADASYN** for oversampling if I were following what I'd tried on a tabular dataset before — it synthesizes new minority rows by finding each minority row's nearest neighbors and interpolating between them. But that nearest-neighbor search is a poor fit for TF-IDF's thousands of sparse, mostly-zero dimensions: "nearest" in that space mostly picks up noise, not meaningful similarity between tweets. So I compared class-weighting against `RandomOverSampler` instead, which just duplicates existing minority rows — a cruder but much more reliable baseline for text.

▶ Code

	approach	f1_macro_mean	accuracy_mean
0	Plain Logistic Regression	0.745832	0.877463
1	Logistic Regression + RandomOverSampler	0.776331	0.850637
2	Logistic Regression (class_weight=balanced)	0.767775	0.841814

▶ Code



Oversampling wins here. This is a different result than I expected going in — I'd assumed class-weighting would be the safer, more reliable default, and I was a little surprised to see plain duplication of minority rows beat it. It's not a contradiction of anything, though: it's exactly why I measured both directly instead of assuming one technique would generalize from one problem to the next. What helps depends on the data and the feature space, and a high-dimensional sparse TF-IDF space clearly behaves differently from a handful of tabular numeric columns.

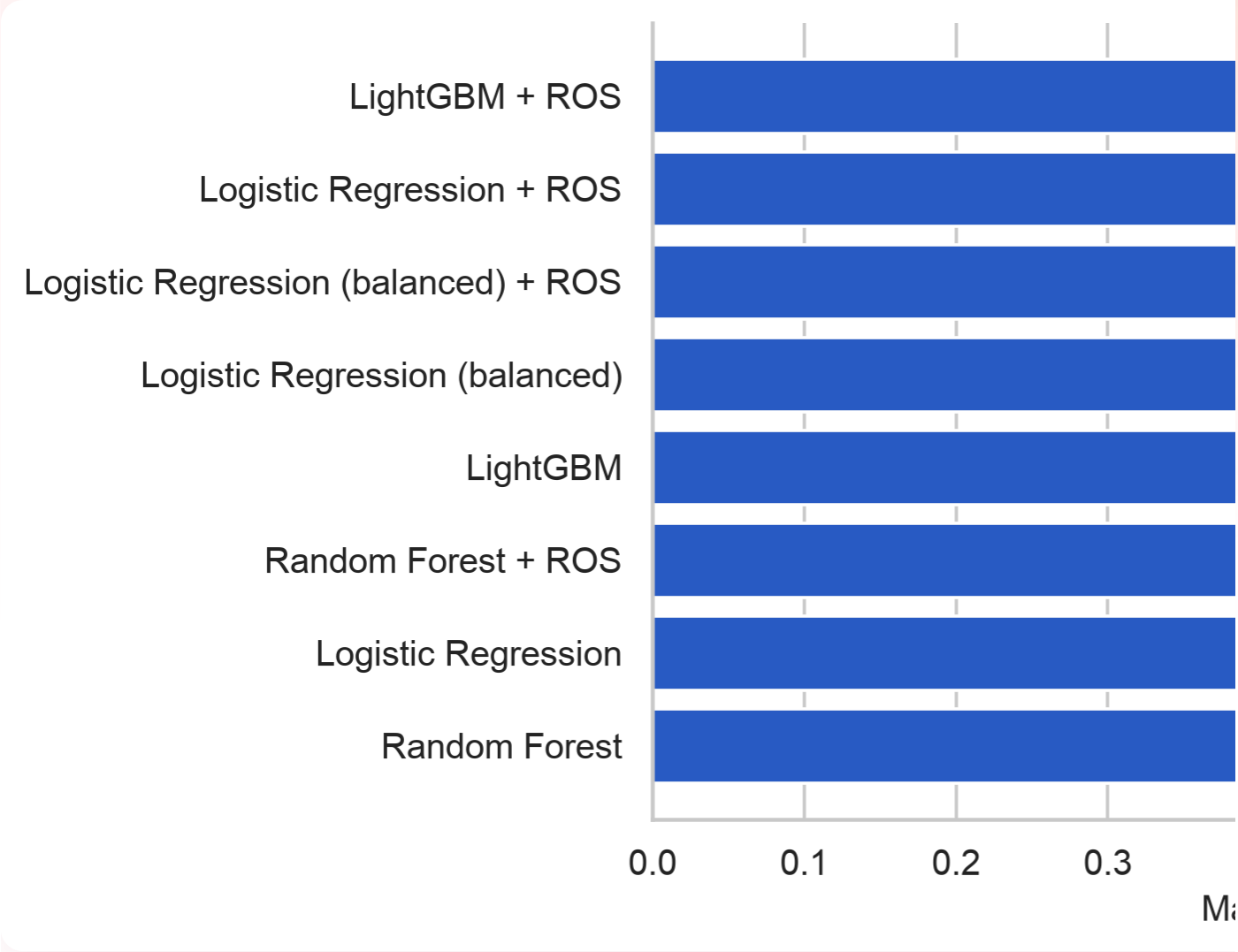
4.3 Model comparison — and a constraint beyond macro-F1

▶ Code

	model	f1_macro_mean	accuracy_mean
7	LightGBM + ROS	0.782396	0.865698
1	Logistic Regression + ROS	0.776331	0.850637
3	Logistic Regression (balanced) + ROS	0.776331	0.850637
2	Logistic Regression (balanced)	0.767775	0.841814
6	LightGBM	0.767339	0.877016
5	Random Forest + ROS	0.750955	0.857053
0	Logistic Regression	0.745832	0.877463

	model	f1_macro_mean	accuracy_mean
4	Random Forest	0.550531	0.834953

▶ Code



`LightGBM + RandomOverSampler` wins here on macro-F1 — but this is where my actual methodology diverges from "just pick the highest number in the table," and this divergence is the single most important decision in this project.

When I first ran this comparison, an *uncapped*-depth Random Forest scored ~0.79 macro-F1 — higher than anything shown in the table above, and higher than the LightGBM model I ended up shipping. My first reaction was that I'd found my winner. But when I tried to explain that Random Forest's predictions with SHAP (see the Model interpretability section below), it became clear why the score looked so good: the model had grown trees to an average depth of ~330, almost entirely by memorizing `RandomOverSampler`'s exact-duplicate minority rows rather than learning generalizable patterns. That depth made `shap.TreeExplainer` take over 2 minutes to compute

values for just 30 rows — completely impractical for anything I'd want to use interactively, let alone put behind an API that needs to explain a prediction on demand.

So I capped that Random Forest's depth to 20 (`model.random_forest_estimator`), which is a deliberate, documented choice rather than a default. That brought it down to ~0.75 macro-F1 — still a reasonable score, but no longer the best one. LightGBM, in contrast, controls its tree growth with `num_leaves` (default 31) rather than needing an explicit depth cap, so it naturally avoids the memorization problem: it reaches the best macro-F1 in the comparison **and** a SHAP call fast enough to use interactively (~0.5s for 300 rows). That's the model I actually trained on the full dataset and saved for the app — chosen for being both accurate and explainable, not for topping a leaderboard.

Why this matters beyond this one project: a model nobody can explain is a liability the moment something goes wrong with it, however good its raw score looks. If a "Disaster" alert gets sent out and a person downstream asks *why* the model flagged that tweet, "I don't know, and computing an answer takes two minutes per prediction" is not a usable answer for a real system — whereas a fast, reliable SHAP explanation is something a human reviewer can actually check in real time. Picking the highest cross-validation number without checking whether the model is explainable is, in effect, optimizing for a number nobody downstream can act on.

5 Choosing a decision threshold

▶ Code

	precision	recall	f1-score	support
Not Disaster	0.939	0.862	0.899	9152
Disaster	0.552	0.752	0.637	2069
accuracy			0.842	11221
macro avg	0.746	0.807	0.768	11221
weighted avg	0.868	0.842	0.851	11221

A classifier like this doesn't just output "Disaster" or "Not Disaster" — it outputs a probability, and a **decision threshold** is the cutoff probability above which I call a tweet "Disaster." The default of 0.5 is just a convention, not something the model itself requires; nothing stops me from lowering it to catch more real disasters at the cost of more false alarms, or raising it to be more conservative. At the default threshold, Disaster recall is ~75% (I catch about three-quarters of real disaster tweets) and precision is ~55% (a bit better than a coin flip on which flagged tweets are real) — a genuinely usable detector, though far from perfect. Since the threshold is a free choice rather than

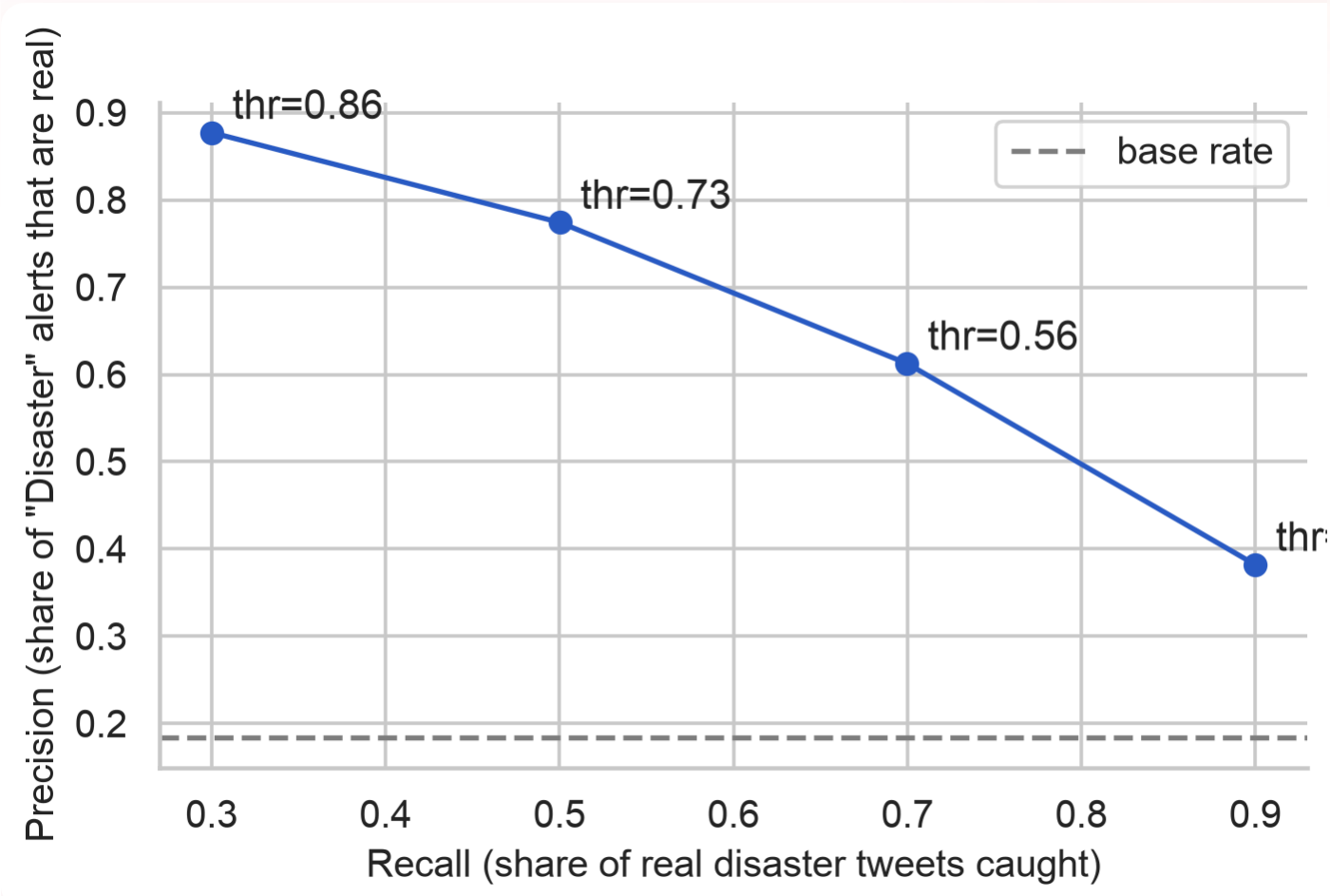
something fixed by the model, the next step sweeps it directly to see what the tradeoff actually looks like at other cutoffs.

▶ Code

```
Average precision (PR-AUC): 0.7143  (base rate: 0.1844)
```

	target_recall	threshold	precision	recall
0	0.3	0.857720	0.877119	0.300145
1	0.5	0.733992	0.774121	0.500242
2	0.7	0.559348	0.612521	0.699855
3	0.9	0.300029	0.382105	0.899952

▶ Code



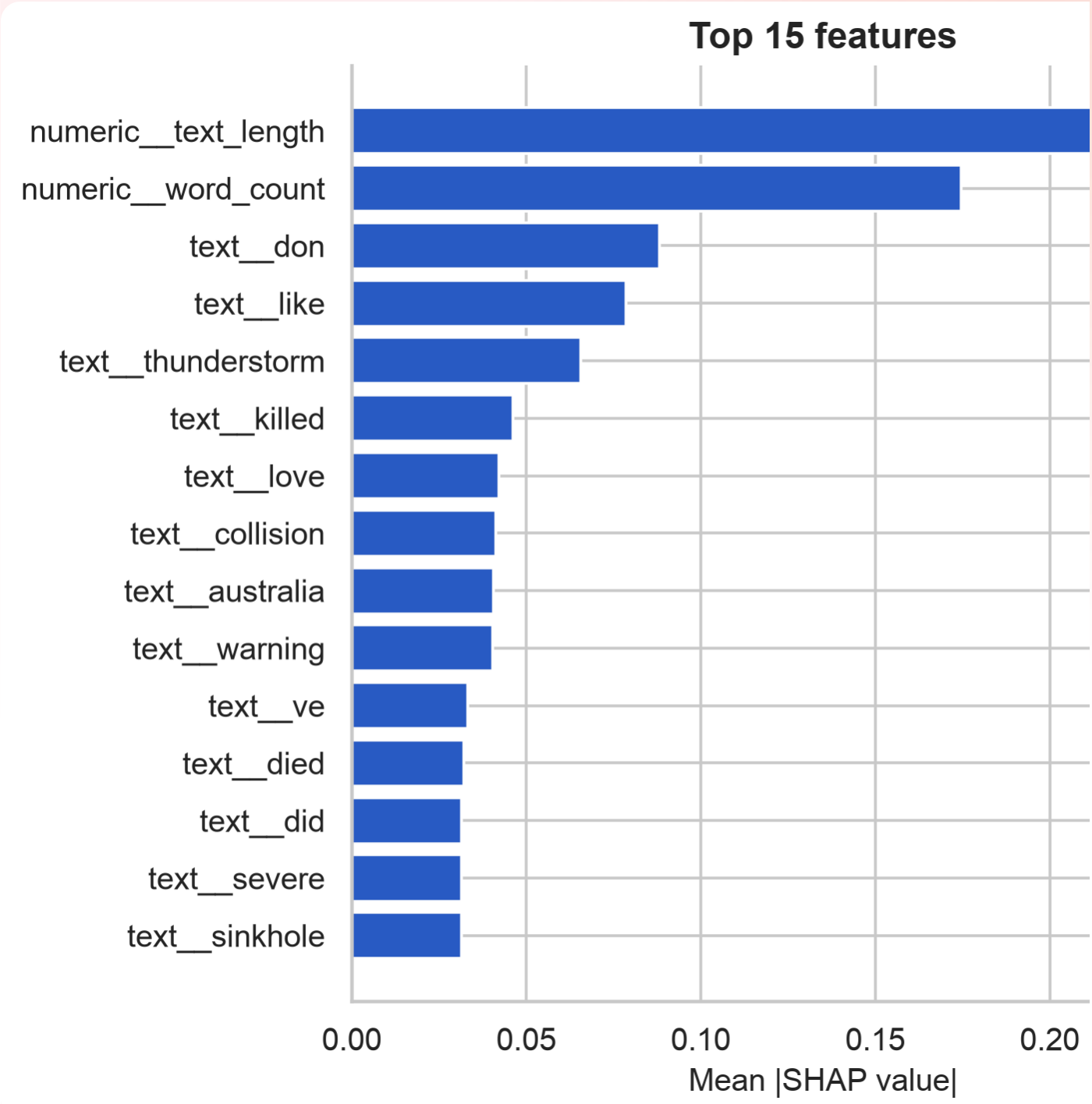
PR-AUC (average precision, computed by averaging precision across every possible threshold) summarizes the whole precision/recall tradeoff in one number, rather than reporting just one cutoff. The PR-AUC here (~0.71) is roughly **3.9x the base rate** (~0.18) — which is my shorthand for

“how much better than blind guessing is this curve, overall.” That’s a stronger lift than I expected walking in. Even at 90% recall (catching 9 out of 10 real disaster tweets), precision stays above 38%, which tells me the tweet text itself genuinely carries a lot of usable signal for this task, more than I’d have guessed before looking at the numbers.

6 Model interpretability

I used **SHAP** (SHapley Additive exPlanations) to understand what the final LightGBM model is actually paying attention to, rather than trusting it as a black box just because its score was good. SHAP assigns each feature a contribution value for each individual prediction — how much that specific feature pushed the prediction toward “Disaster” or away from it — based on game-theory ideas about fairly splitting credit among contributors. For a tree-based model like LightGBM, `shap.TreeExplainer` computes these values by walking the model’s actual trees, which is what makes it fast enough to run interactively here (as opposed to a model-agnostic explainer, which would need to re-run the model many times per prediction and would have been far too slow for the same reason the uncapped Random Forest was impractical above). Averaging the absolute SHAP value of each feature across many tweets gives a ranking of which features matter most overall, which is what the chart below shows.

▶ Code



`text_length` and `word_count` dominate every other feature by a wide margin, which genuinely surprised me — I expected specific disaster-related words to lead the ranking, not two simple counts I'd almost skipped adding. My best explanation: real disaster tweets (often news-style reports) tend to be longer and more detail-dense than casual tweets that merely use disaster-adjacent words figuratively (e.g. "this traffic is a disaster"). Beyond those two, specific words do carry real signal too (`killed`, `collision`, `warning`, place names), but they matter much less than the two structural, hand-crafted numeric features. This is a finding I wouldn't have without

deliberately engineering those simple counts alongside the TF-IDF vocabulary — if I'd relied on TF-IDF word features alone, I'd have missed the single strongest signal in the whole model.

7 Discussion: what this project doesn't do

I classified individual tweets as disaster-related or not — I want to be upfront that this is a genuinely narrower problem than a real disaster-response or emergency-triage system would need to solve:

- **No real-time stream processing.** My model scores tweets one at a time, in batch. An actual monitoring system needs to ingest a live stream, deduplicate near-identical retweets/reposts (a much harder version of the exact-duplicate check I did here), and rate-limit alerts so a viral non-disaster tweet doesn't flood a response team.
- **No geolocation.** `has_location` is the only signal I kept from the `location` field, specifically because the raw text was too inconsistent to parse reliably. A real system would need proper geocoding — resolving free-text location mentions to actual coordinates — which is a substantial NLP problem of its own, not something I attempted here.
- **No verification or severity ranking.** My model answers "is this about a disaster," not "is this true" or "how severe is it." Both are necessary next questions before anyone could act on an alert from this model.
- **No temporal/event clustering.** Real disaster response cares about detecting an *emerging event* — a spike in related tweets about the same incident — not just classifying tweets independently. That's a clustering/trend-detection problem layered on top of the per-tweet classification I built, and it's not something my current architecture does at all.

None of this is a limitation specific to how I executed this project — it's the honest scope boundary between "classify this tweet's topic" and "build a disaster-response monitoring system," and I'd rather state that boundary plainly than let the model's decent numbers imply something bigger than what it actually does.

8 Conclusion

Working through a moderately imbalanced, real text-classification dataset gave me a few honest, measured findings I hadn't fully appreciated before starting: accuracy overstates performance here, just as I'd expect on any imbalanced dataset, even though the imbalance itself is moderate rather than extreme; oversampling beat class-weighting for TF-IDF features, which was a different result than I expected going in; and the strongest-scoring model I measured wasn't the one I

shipped, because scoring highest on macro-F1 and being practical to explain with SHAP turned out to be in direct tension with each other. Choosing the production model on that basis — not on the single highest number in a sweep — is the main methodological point I wanted this project to make, and I think it generalizes well beyond this one dataset: a model's raw score is only half the story if nobody can explain why it made a given prediction. Real disaster-response triage remains a substantially larger problem than what I built here, and I'd rather state that scope boundary plainly than blur it.